

1. Database Schema & Normalization (P1)

To achieve **BCNF** (Boyce-Codd Normalization Form) and ensure scale:

- **1NF**: All column values are atomic (e.g., separating `seat_row` and `seat_number`).
- **2NF**: Every non-key attribute fully depends on the primary key (no partial dependencies on composite keys).
- **3NF & BCNF**: No transitive dependencies (e.g., `theatre_id` is linked to `screens`, while `screens` is linked to `shows`, removing redundancy from `shows`).

SQL

-- 1. Users Table

```
CREATE TABLE users (  
  user_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  phone_number VARCHAR(15) UNIQUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

-- 2. Movies Table

```
CREATE TABLE movies (  
  movie_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  language VARCHAR(50) NOT NULL,  
  genre VARCHAR(50),  
  duration_minutes INT NOT NULL,  
  rating VARCHAR(10)  
) ENGINE=InnoDB;
```

-- 3. Theatres Table

```
CREATE TABLE theatres (  
  theatre_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(150) NOT NULL,  
  city VARCHAR(100) NOT NULL,  
  location VARCHAR(255) NOT NULL,  
  total_screens INT DEFAULT 1  
) ENGINE=InnoDB;
```

-- 4. Screens Table

```
CREATE TABLE screens (  
  screen_id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
theatre_id BIGINT NOT NULL,  
screen_name VARCHAR(50) NOT NULL,  
total_seats INT NOT NULL,  
FOREIGN KEY (theatre_id) REFERENCES theatres(theatre_id) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

-- 5. Physical Seats Table

```
CREATE TABLE seats (  
    seat_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    screen_id BIGINT NOT NULL,  
    seat_row VARCHAR(5) NOT NULL,  
    seat_number INT NOT NULL,  
    seat_type ENUM('REGULAR', 'VIP', 'RECLINER') DEFAULT 'REGULAR',  
    FOREIGN KEY (screen_id) REFERENCES screens(screen_id) ON DELETE CASCADE,  
    UNIQUE KEY uq_screen_seat (screen_id, seat_row, seat_number)  
) ENGINE=InnoDB;
```

-- 6. Shows Table (Relates Movie, Screen, Date, and Time)

```
CREATE TABLE shows (  
    show_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    movie_id BIGINT NOT NULL,  
    screen_id BIGINT NOT NULL,  
    show_date DATE NOT NULL,  
    start_time TIME NOT NULL,  
    end_time TIME NOT NULL,  
    price DECIMAL(10, 2) NOT NULL,  
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id),  
    FOREIGN KEY (screen_id) REFERENCES screens(screen_id),  
    INDEX idx_show_search (screen_id, show_date)  
) ENGINE=InnoDB;
```

-- 7. Show Seats Table (Tracks Availability per Show & Handles Concurrency)

```
CREATE TABLE show_seats (  
    show_seat_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    show_id BIGINT NOT NULL,  
    seat_id BIGINT NOT NULL,  
    status ENUM('AVAILABLE', 'LOCKED', 'BOOKED') DEFAULT 'AVAILABLE',  
    locked_until TIMESTAMP NULL,  
    FOREIGN KEY (show_id) REFERENCES shows(show_id) ON DELETE CASCADE,  
    FOREIGN KEY (seat_id) REFERENCES seats(seat_id),  
    UNIQUE KEY uq_show_seat (show_id, seat_id),  
    INDEX idx_concurrency_check (show_id, status, locked_until)  
) ENGINE=InnoDB;
```

-- 8. Bookings Table

```
CREATE TABLE bookings (  
    booking_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    user_id BIGINT NOT NULL,  
    show_id BIGINT NOT NULL,  
    total_amount DECIMAL(10, 2) NOT NULL,  
    booking_status ENUM('PENDING', 'CONFIRMED', 'CANCELLED', 'EXPIRED') DEFAULT  
'PENDING',  
    payment_id VARCHAR(100) UNIQUE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (show_id) REFERENCES shows(show_id)  
) ENGINE=InnoDB;
```

2. Sample Data Insertion (P1)

-- Insert Sample User

```
INSERT INTO users (full_name, email, phone_number) VALUES  
('Rahul Sharma', 'rahul@example.com', '9876543210');
```

-- Insert Sample Movie

```
INSERT INTO movies (title, language, genre, duration_minutes, rating) VALUES  
('Inception', 'English', 'Sci-Fi', 148, 'UA');
```

-- Insert Sample Theatre & Screen

```
INSERT INTO theatres (name, city, location, total_screens) VALUES  
('PVR Koramangala', 'Bangalore', 'Forum Mall, Koramangala', 2);
```

```
INSERT INTO screens (theatre_id, screen_name, total_seats) VALUES  
(1, 'Screen 1', 2);
```

-- Insert Sample Seats

```
INSERT INTO seats (screen_id, seat_row, seat_number, seat_type) VALUES  
(1, 'A', 1, 'VIP'),  
(1, 'A', 2, 'VIP');
```

-- Insert Sample Show

```
INSERT INTO shows (movie_id, screen_id, show_date, start_time, end_time, price) VALUES  
(1, 1, '2026-10-01', '18:30:00', '21:00:00', 350.00);
```

-- Initialize Show Seats Status

```
INSERT INTO show_seats (show_id, seat_id, status) VALUES  
(1, 1, 'AVAILABLE');
```

```
(1, 2, 'AVAILABLE');
```

3. Concurrency & Locking Strategy

To handle thousands of users competing for seats without double-booking:

1. Redis Temporary Locks (Primary Layer):

- When a user selects a seat, issue a Redis distributed lock (**SETNX** with a TTL of 10 minutes): **SET lock:show:1:seat:1 "user_123" EX 600 NX**
- If Redis returns **OK**, proceed to mark the database row as **LOCKED** with a **locked_until** timestamp.

2. Database Pessimistic Locking (Safety Layer):

- When processing the transaction in MySQL, query using **FOR UPDATE** to lock the rows during the transaction block:

```
START TRANSACTION;
```

```
-- Select and lock available or expired-lock seats
```

```
SELECT show_seat_id FROM show_seats
```

```
WHERE show_id = 1 AND seat_id IN (1, 2)
```

```
AND (status = 'AVAILABLE' OR (status = 'LOCKED' AND locked_until < NOW()))
```

```
FOR UPDATE;
```

```
-- Mark as LOCKED for 10 minutes
```

```
UPDATE show_seats
```

```
SET status = 'LOCKED', locked_until = NOW() + INTERVAL 10 MINUTE
```

```
WHERE show_id = 1 AND seat_id IN (1, 2);
```

```
COMMIT;
```

4. Fetch Shows for a Given Theatre & Date (P2)

```
SELECT
  t.name AS theatre_name,
  m.title AS movie_title,
  m.language,
  m.rating,
  s.screen_name,
  sh.show_id,
  sh.show_date,
  sh.start_time,
  sh.end_time,
  sh.price
FROM shows sh
JOIN screens s ON sh.screen_id = s.screen_id
JOIN theatres t ON s.theatre_id = t.theatre_id
JOIN movies m ON sh.movie_id = m.movie_id
WHERE t.theatre_id = 1
      AND sh.show_date = '2026-10-01'
ORDER BY m.title, sh.start_time;
```

5. Entity & Cardinality Summary

- **THEATRES** → **SCREENS** (1 : N): A theatre can have multiple screens; each screen belongs to exactly one theatre.
- **SCREENS** → **SEATS** (1 : N): A screen contains multiple physical seats.
- **SCREENS & MOVIES** → **SHOWS** (1 : N): A screen hosts multiple shows across dates/times, and a movie can be scheduled for multiple shows.
- **SHOWS & SEATS** → **SHOW_SEATS** (N : M mapping table): Tracks dynamic seat status (**AVAILABLE**, **LOCKED**, **BOOKED**) and timeout (**locked_until**) per individual show to prevent double-booking under high concurrency.
- **USERS & SHOWS** → **BOOKINGS** (1 : N): A user can make multiple bookings; each booking is associated with a specific show.